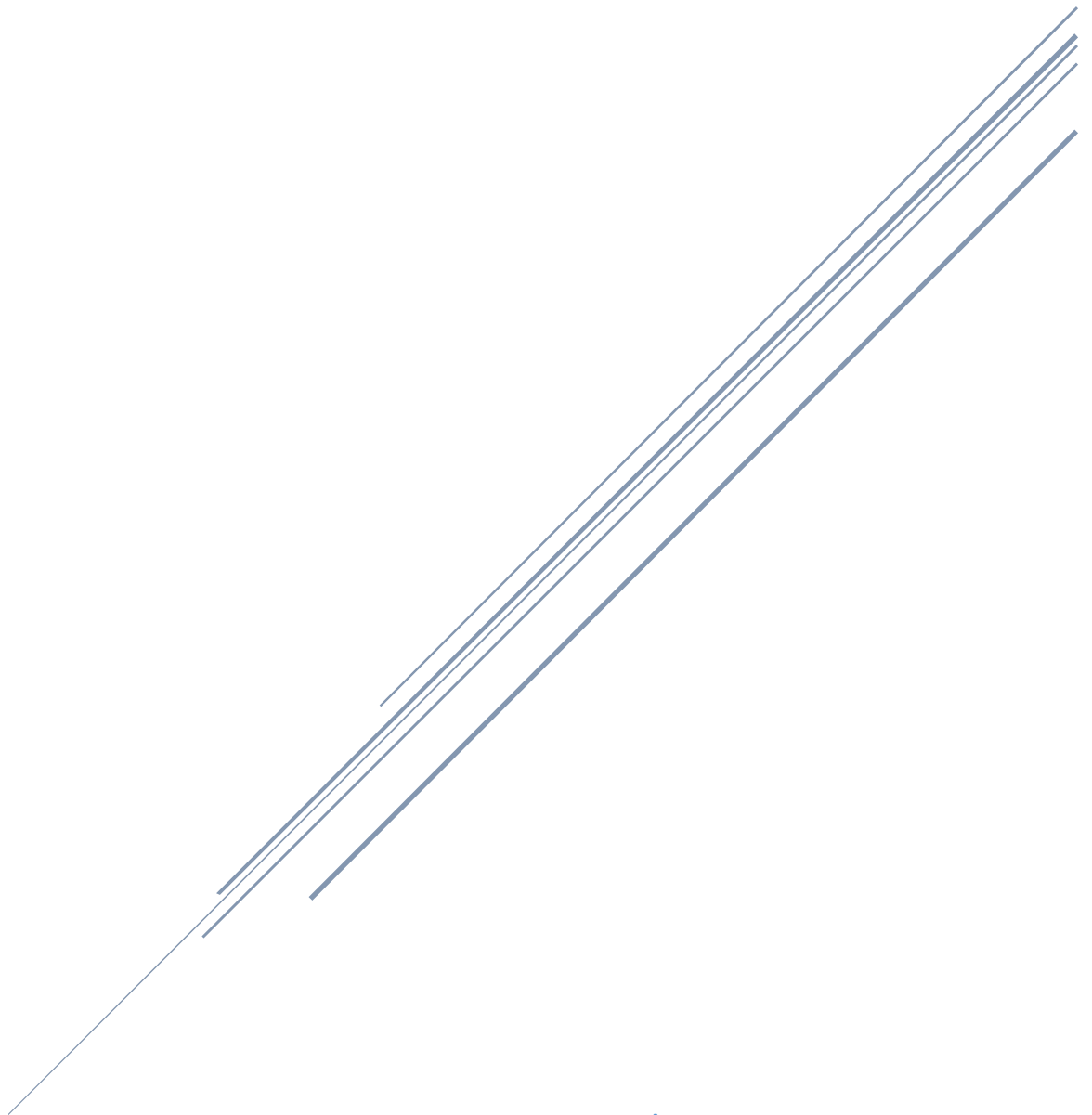


CODE AND SUMMARY OF CLASS 15

Sahir Ahmed Sheikh

Saturday (2 – 5)



Teachers:

Muhammad Bilal And Ali Aftab Sheikh

Code And Summary Of Class 15 – Saturday (2 – 5) | Quarter 3

Introduction

This document provides a detailed summary of Class 15 of Quarter 3, held on Saturday from 2 to 5 PM, led by Sir Ali Aftab Sheikh. In this session, Muhammad Umar, the CR from Sir Hamzah Syed's class, also joined us. He will also be teaching us in today's session. The session focused on revising AI agent concepts, **tools**, **agent-as-tool**, and **handoff** theory, with practical coding demonstrations. The class built on previous topics like Chainlit, chatbots, streaming, and tools, emphasizing practical application and debugging. Mohammed Umar provided an in-depth explanation of tool parameters, agent loop, max turns, and visualization. The session included theoretical discussions, code examples, and motivational guidance on focusing on one skill for career growth and preparing for Quarter 4's advanced topics.

The class addressed the following key objectives:

- Reviewing and checking assignments from previous classes, with a focus on filtering non-serious students due to heat and class merging.
- Revising tools, agent-as-tool, and handoff concepts with practical examples.
- Encouraging students to experiment with code parameters, max turns, and visualization for better understanding.
- Providing motivational advice on skill specialization, consistent learning, and avoiding distractions from short-term success stories.
- Highlighting the importance of practical coding, source code analysis, and professional development practices for industry relevance.

Students were reminded of the upcoming test, strict assignment policy, and the need to push code to GitHub. The session stressed self-directed learning and preparation for Quarter 4's challenges.

Topics Covered

1. Class Updates and Quarter 3 Progress

- **Assignment Check and Filtering:** Reviewed three assignments from previous classes. Checked completion status, with focus on boys and girls. Introduced filtering due to heat and class merging if students are not serious.
- **Motivational Advice:** Emphasized focusing on one skill for career growth, avoiding multi-field distractions. Shared examples from Amazon VA courses and the importance of consistent income over one-time hits.
- **Test Preparation:** Upcoming test in early/mid-August with practical questions. Advised reviewing documentation, running code, and experimenting with parameters.
- **Class Merging Policy:** If assignments remain incomplete, students risk elimination or merging with another class. CRs and assistants will check assignments next week.

2. Revision Of AI Agent Concepts:

The session revised key agent concepts, emphasizing tools, handoff, and agent-as-tool. Key points included:

- **Tools:** Essential for tasks LLMs can't handle alone, like real-time data. Difference from handoff: tools return to LLM for final output; handoff transfers full control.
- **Agent-as-Tool:** Converts agents to tools for specific tasks without transferring conversation history. Used for multi-agent systems like translators.
- **Handoff:** Transfers entire conversation and control to another agent for specialized handling.
- **LLM Concepts:** Stateless nature, conversation history, hallucination, and cutoff knowledge. Explained with examples like weather queries and web search tools.

3. AI Architecture: Tools, Agent-as-Tool, & Handoff Theory

3.1 Tools

***Definition and Purpose:** Tools are extra functions that an AI can use to do tasks it normally can't, like getting live information (for example, the weather). They help the AI avoid mistakes and deal with old or missing knowledge.*

Key Features:

- **Function Calling:** Tools are defined as Python functions for specific tasks.
- **Custom vs. Hosted:** Custom (developer-defined, e.g., API calls); Hosted (pre-built, e.g., Open AI web search).
- **Integration:** Registered with agents; LLM decides invocation based on query.

How It Works:

- Four items to LLM: System prompt, user prompt, tool message, tool schema.
- Agent loop: LLM returns final output or tool call; tool response goes back to LLM (default) or directly to user.
- Max Turns: Limits loop iterations to prevent infinite loops; default 10.

Practical Application:

- In class, demonstrated student finder tool to fetch data by roll number.

- **Example:**

```
import os
from openai_agents import Agent, Runner
from typing import cast

load_dotenv()

api_key = os.getenv("GEMINI_API_KEY")
if not api_key:
    raise ValueError("Gemini API key is not set. Please ensure it's defined in your .env")

@async def main():
    external_client = AsyncOpenAI(base_url="https://api.gemini.com/v1", api_key=GEMINI_API_KEY)
    run_config = {"model": "gemini-1.5-pro", "external_client": external_client, "tracing": True}

    agent = Agent(name="Student Finder", instructions="You are a helpful assistant.")
    user_session.set("agent", agent)

    await cl.Message(content="Welcome to the Assistant! How can I help you?").send()

@async def on_message(message: cl.Message):
    run_config = cast(dict, user_session.get("run_config"))
    agent = cast(Agent, user_session.get("agent"))
    chat_history = user_session.get("chat_history", [])
    chat_history.append({"role": "user", "content": message.content})

    try:
        response = Runner.run_sync(agent, message.content, run_config=run_config)
        chat_history.append({"role": "assistant", "content": response.final_output})
        user_session.set("chat_history", chat_history)
        await cl.Message(content=response.final_output).send()
    except Exception as e:
        await cl.Message(content=f"Error: {str(e)}").send()

cl.run()
```

- **Explanation:** Creates agent with name, instructions, tool. Runs with max turns to show turns and errors if limit exceeded.

Why Its Important:

- Enables accurate, real-time actions in AI systems.
- Prepares for Quarter 4's advanced multi-agent applications.

3.2 Agent-as-Tool

***Definition and Purpose:** Converts an agent into a tool for specific tasks without transferring control or conversation history. Used in multi-agent systems for modular functionality.*

Key Features:

- **No Control Transfer:** Response returns to main agent; conversation continues with original agent.
- **Difference from Handoff:** Handoff transfers history and control; agent-as-tool only processes generated input.
- **Specialization:** Ideal for tasks like translation in specific languages.

How It Works:

- Use `.as_tool` on agent to convert.
- Main (triage) agent calls tools based on instructions (e.g., "Always use provided tools").
- Example: Translator agents for Spanish, French, Italian as tools.

Practical Application:

- Built translator chatbot with triage agent managing language-specific tools.

```
import os, chainlit as cl
from openai_agents import Agent, Runner
from dotenv import load_dotenv

load_dotenv()
api_key = os.getenv("GEMINI_API_KEY")
if not api_key: raise ValueError("Gemini API key is not set.")

@cl.on_chat_start
async def start():
    client = AsyncOpenAI(base_url="https://api.gemini.com/v1", api_key=api_key)
    run_cfg = {"model": "gemini-1.5-pro", "external_client": client}

    langs = {"Spanish": "translate_to_spanish", "French": "translate_to_french", "Italian": "translate_to_italian"}
    tools = [Agent(name=f"({l}) Translator", instructions=f"Translate to ({l}).").as_tool(name=t, description=f"To ({l})")
             for l, t in langs.items()]

    triage = Agent(name="Triage Agent", instructions="Use tools for translation only.", tools=tools)
    cl.user_session.set("triage", triage), cl.user_session.set("cfg", run_cfg), cl.user_session.set("hist", [])
    await cl.Message(content="Welcome! Enter text to translate.").send()

@cl.on_message
async def on_msg(msg: cl.Message):
    triage, run_cfg, hist = cl.user_session.get("triage"), cl.user_session.get("cfg"), cl.user_session.get("hist")
    hist.append({"role": "user", "content": msg.content})
    thinking = await cl.Message(content="Thinking...").send()
    try:
        res = Runner.run_sync(triage, msg.content, run_config=run_cfg)
        hist.append({"role": "assistant", "content": res.final_output})
        cl.user_session.set("hist", hist)
        await thinking.update(content=res.final_output)
    except Exception as e:
        await thinking.update(content=f"Error: {e}")
```

- You can zoom in and see.
- **Explanation:** Triage agent uses tools for translations; refuses unsupported languages.

Why Its Important:

- Enables modular, scalable AI systems without conflicts.
- Essential for Quarter 4's focus on collaborative agents.

3.3 Handoff Theory

***Definition and Purpose:** Transfers full control and conversation history to another agent for specialized handling.*

Key Features:

- **Full Transfer:** New agent continues conversation independently.
- **Use Cases:** Routing tasks to expert agents (e.g., design to designer agent).

How It Works:

- Main agent calls handoff; control shifts completely.
- Difference from Tools/Agent-as-Tool: No return to main agent; new agent returns final output.

Practical Application:

- Discussed theoretically; code to be covered next class.

Why Its Important:

- Supports complex, multi-agent workflows for real-world applications.

4. Practical Coding and Debugging

- **Environment Setup:** UV project, install openai-agents, chainlit, python-dotenv. Load Gemini API key via .env.
 - **Debugging:** Check required parameters (e.g., name), source code for asterisks (indicating required). Enable logging for turns/tokens; visualize agent flow with graph library.
 - **Validation Logic:** Experiment with max turns, empty strings; avoid sensitive data exposure.
-

5. Key Insights and Takeaways

- **AI Agent Development:** Master tools, agent-as-tool, handoff for practical AI.
 - **Self Learning:** Experiment with code, review source code/GitHub for test prep.
 - **Industry Relevance:** Focus on one skill, consistent practice for IT careers.
 - **Professional Coding Practices:** Use logging/visualization for observability; push to GitHub.
-

6. 📖 Class Resources and Assignments

- **Today's Class Code**
Students are encouraged to experiment with the provided examples to deepen understanding.
 - **Complete Previous Assignments:**
Ensure that the previous assignments are completed and pushed to GitHub, and submit the link via the [assignment submission form](#).
-

Important Reminder

- Use documentation and tools like NotebookLM for test preparation, focusing on practical coding skills.
 - All Agents projects must be on GitHub.
 - Stay active on Discord for class announcements and review the [Panaversity](#) repository for the syllabus.
-

Final Words

Your responsibility is to learn actively, participate confidently, and practice consistently. Quarter 4 will demand greater effort with advanced topics like AI and cloud computing. Embrace challenges, complete assignments, and build your portfolio to seize IT industry opportunities. Stay focused, and may Allah guide you.

Allah Hafiz – See you in the Next Class!

Repo Link: [GIAIC-Sat-Afternoon](#)

This is my repository where I have uploaded all the class codes, assignments, and detailed summaries. You can go through each class code and its full detailed summary without any hesitation, and it will also help you prepare well for your test.

Thank You for Reading!

Hope you understood Class 15 well.

"Real learning isn't about memorizing facts, it's about awakening the mind – to think, to imagine, and to transform the future." – *Sahir Ahmed Sheikh*